

COMP532- Assignment 2 Deep Reinforcement Learning

Group Members and Contributions

Name	Student ID	Email	Contribution
Vishal Saravanan	201900556	v.saravanan@liverpool.ac.uk	Lunar Lander
Laura Valentina Sierra Pena	201936067	sglsierr@liverpool.ac.uk	Chess Agent
Ainur Smailova	201874285	sgasmai2@liverpool.ac.uk	Exploration-Exploitation & Reporting
Saif Ur Rehman	201940185	sgsrehm1@liverpool.ac.uk	Humanoid Implementation

Problem 1: Deep Reinforcement Learning Agent

Introduction

This report presents the implementation and evaluation of deep reinforcement learning agents for the LunarLander-v3 environment from Gymnasium. Three algorithms were implemented and compared:

- Deep Q-Network (DQN V1 - baseline)
- Improved Deep Q-Network (DQN V2)
- Proximal Policy Optimisation (PPO V3)

The goal is to analyse learning performance, stability, and the impact of exploration strategies.

A GIF demonstrating the trained agent's behaviour in the LunarLander-v3 environment is included as a separate file (`lunar_lander_agent.gif`) in the submission.

DQN V1 (Baseline)

The baseline DQN showed steady and consistent learning behaviour. The environment was solved at episode 947, with a final average score of 217.39.

Key observations:

- Stable training with no major instability
- Slow convergence
- Continued improvement after solving threshold

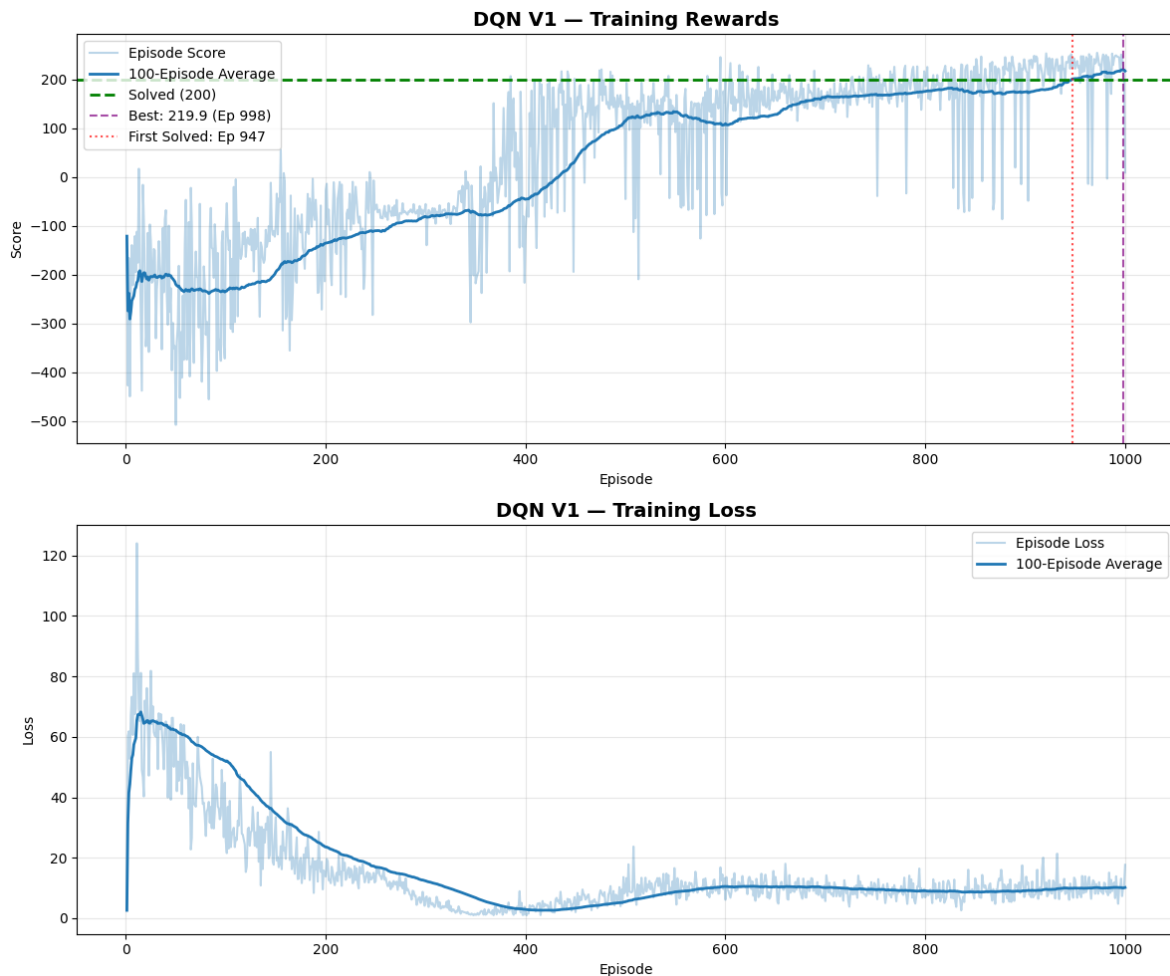


Figure 1: DQN V1 training curves (episodes 0–1000). Top: episode reward with 100-episode rolling average and solved threshold (200). Bottom: training loss with 100-episode rolling average. First solved at episode 947; best score 219.9 at episode 998.

Figure 1 shows the full training progression of DQN V1 over 1000 episodes. In the reward plot (top panel), the per-episode scores (light blue) exhibit very high variance throughout training, which is typical of DQN on continuous control tasks where small changes in behaviour can lead to dramatically different outcomes. The 100-episode rolling average (dark blue) provides a clearer picture of the underlying learning trend: it starts deeply negative (around -200) in the first 50 episodes as the agent acts almost randomly, then rises slowly and consistently, crossing zero around episode 250 and reaching the solved threshold of 200 only at episode 947. This slow but monotonic improvement is characteristic of conservative epsilon decay, which reduces exploration quickly and results in stable but slow learning. The best individual episode score of 219.9 is achieved near episode 998, close to the end of training.

The training loss plot (bottom panel) tells a complementary story. The loss peaks sharply at around 120 in the first 30 episodes, reflecting the unstable early phase when the replay buffer is sparsely populated and Q-value estimates are highly inaccurate. It then decays steadily to approximately 10 by episode 200 and stabilises at this low level for the remainder of training, indicating that the Q-network has converged to a consistent representation of the value function. The absence of any secondary loss spikes after episode 200 is a strong indicator of training stability — a key strength of the DQN V1 baseline.

DQN V2 (Improved)

DQN V2 was developed by tuning key hyperparameters to improve convergence speed.

Hyperparameter Changes

Parameter	DQN V1	DQN V2	Reason
BUFFER SIZE	100,000	150,000	More diverse experience
Learning Rate	0.0005	0.0003	Stable updates
TAU	0.001	0.005	Faster target updates
Epsilon End	0.01	0.05	More exploration
Epsilon Decay	0.995	0.997	Slower decay

Table 1: DQN Hyperparameter Comparison

Results

Metric	DQN V1	DQN V2
First Solved Episode	947	629
Best Score	219.92	250.09
Final Score	217.39	182.07

Table 2: DQN Performance Comparison

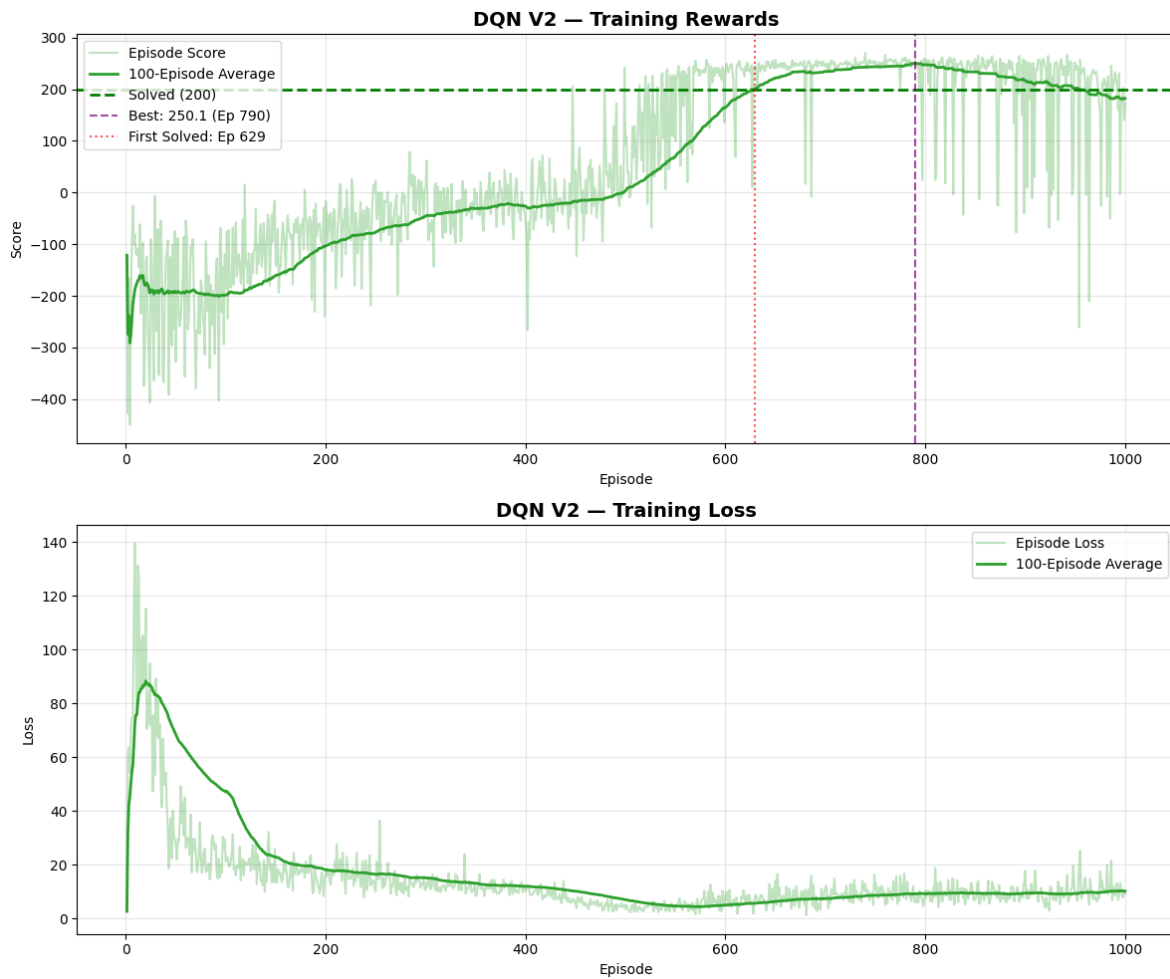


Figure 2: DQN V2 training curves (episodes 0–1000). Top: episode reward with 100-episode rolling average and solved threshold (200). Bottom: training loss. First solved at episode 629; best score 250.1 at episode 790.

Figure 2 illustrates the impact of the improved hyperparameter configuration on both learning speed and stability. The most immediately noticeable difference from DQN V1 is in the reward plot (top panel): the 100-episode rolling average rises considerably faster, crossing the solved threshold of 200 at episode 629 — a 318-episode improvement over the baseline. The rolling average continues to climb, reaching a peak of approximately 250 around episode 790, which corresponds to the best individual score of 250.1 recorded in the entire run. However, a clear degradation in the rolling average is visible after episode 800, where it falls back below 200 near the end of training. This late-stage instability is a direct consequence of the higher minimum epsilon (0.05): because the agent continues to take random actions with 5% probability even after the policy has converged, it periodically makes poor decisions that pull the rolling average downward. This illustrates the core trade-off of aggressive exploration: faster discovery of good policies at the cost of long-term stability.

The training loss plot (bottom panel) shows a broadly similar pattern to DQN V1 but with a slightly higher early peak (approximately 140 at episode 20) and a slower initial decay, reflecting the larger replay buffer and the greater diversity of experiences being sampled. The loss stabilises at approximately 10 from around episode 400 onward, and notably shows a small secondary increase around episodes 800–1000, coinciding precisely with the reward instability observed in the top panel. This correlation confirms that the late-stage performance degradation is a genuine policy effect caused by sustained epsilon exploration rather than a numerical artefact.

Key observations:

- Faster convergence (318 episodes earlier)
- Higher peak performance
- Lower final stability due to sustained exploration

PPO V3

PPO is a policy gradient method that directly optimises the policy using a clipped objective. PPO was trained for 300,000 timesteps — equivalent to approximately 1000 episodes based on an average episode length of 248 steps per episode.

Hyperparameters

Parameter	Value
Learning Rate	0.0003
n_steps	1024
Batch Size	64
Epochs	10
Gamma	0.99
Clip Range	0.1
Entropy Coefficient	0.01

Table 3: PPO Hyperparameters

Results

Metric	Value
Episodes	1207
First Solved	627
Best Score	253.85
Final Score	252.35

Table 4: PPO Performance

Key observations:

- Smooth and stable learning
- No instability observed
- Highest overall performance

Final Comparison

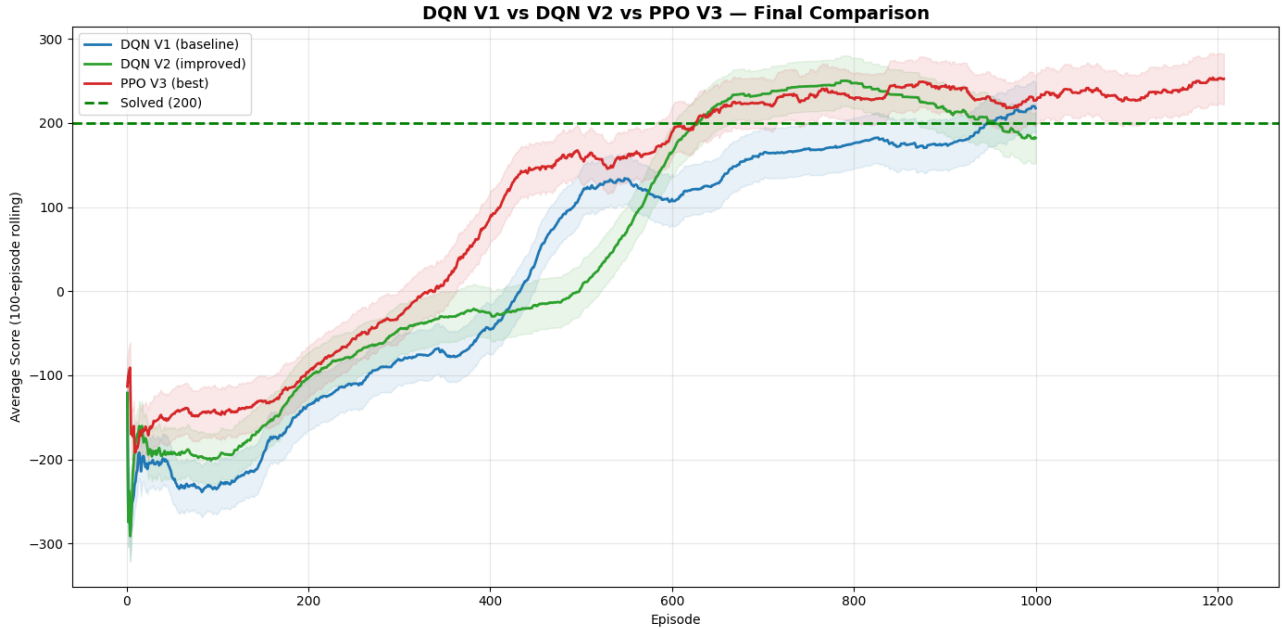


Figure 3: Final comparison of DQN V1, DQN V2, and PPO V3: 100-episode rolling average reward with shaded variance bands from episode 0 to 1200. The dashed green line marks the solved threshold (200). PPO V3 achieves the earliest stable convergence above the threshold.

Figure 3 places all three algorithms on the same axis, allowing a direct comparison of their learning trajectories. Several important observations emerge from this plot. First, all three methods start from a similar negative baseline around episode 0, confirming that the task is genuinely challenging and requires sustained learning before the agent performs meaningfully. Second, PPO V3 (red) separates from the two DQN variants earliest: its rolling average rises above zero by approximately episode 300 and crosses the solved threshold around episode 627, at which point it continues to increase rather than plateau. This upward trajectory persists well beyond episode 1000, reaching approximately 250 by episode 1200, and crucially does so without the instability that characterises DQN V2 after episode 800.

The shaded variance bands around each curve are also informative. PPO V3 has the narrowest band across the full training run, indicating that its performance is highly consistent across episodes. DQN V2 has a moderate band that widens noticeably after episode 600, while DQN V1 maintains a narrow but consistently lower band. This confirms that the stability advantage of PPO is not merely a difference in final score but reflects a fundamentally more consistent policy throughout training. The entropy-based exploration mechanism in PPO allows it to reduce stochasticity naturally as the policy improves, whereas DQN V2's fixed minimum epsilon forces continued randomness regardless of policy quality, directly causing the variance increase seen late in training.

Algorithm	Type	Solved	Best Score	Final Score
DQN V1	Value-based	947	219.92	217.39
DQN V2	Value-based	629	250.09	182.07
PPO V3	Policy-based	627	253.85	252.35

Table 5: Overall Comparison

Evaluation Results

Metric	DQN V2	PPO V3
Perfect Landings	5%	75%
Crashes	55%	0%
Average Score	121.43	265.45

Table 6: Evaluation over 20 Episodes

Conclusion: PPO V3 demonstrates superior performance in both stability and consistency.

These results highlight the importance of exploration strategies in reinforcement learning, which are analysed in detail in the next section.

Problem 2: Exploration vs Exploitation in Deep Reinforcement Learning

The Exploration–Exploitation Trade-off

In deep reinforcement learning, an agent must learn how to act in an environment by interacting with it and receiving rewards. A fundamental challenge in this process is the trade-off between exploration and exploitation.

Exploration refers to the process of trying new or less-known actions in order to discover potentially better strategies. Exploitation refers to selecting actions that are already known to produce high rewards based on past experience.

These two objectives are in direct conflict. If the agent focuses too much on exploration, it may waste time trying suboptimal actions and learning becomes slow. On the other hand, if the agent exploits too early, it may converge to a suboptimal policy and fail to discover better solutions.

Therefore, a successful reinforcement learning algorithm must balance these two behaviours throughout training.

Exploration in Value-Based Methods (DQN)

In value-based methods such as Deep Q-Network (DQN), exploration is typically implemented using an epsilon-greedy policy. Under this strategy, the agent selects a random action with probability ϵ and selects the action with the highest predicted Q-value with probability $1 - \epsilon$.

At the beginning of training, ϵ is set to a high value (usually 1.0), encouraging full exploration. As training progresses, ϵ is gradually reduced according to a decay schedule, shifting the agent towards exploitation.

This approach provides a simple and effective mechanism for controlling exploration. However, it requires careful tuning of both the decay rate and the minimum value of ϵ , as these directly influence learning behaviour.

Effect of Exploration in DQN V1 and DQN V2

The experimental results from Problem 1 clearly demonstrate the impact of different exploration strategies in DQN.

DQN V1 uses a faster epsilon decay (0.995) and a lower minimum epsilon (0.01). As a result, exploration is reduced relatively quickly, and the agent becomes more deterministic earlier in training. This leads to slower convergence, with the environment being solved at episode 947.

In contrast, DQN V2 uses a slower decay rate (0.997) and a higher minimum epsilon (0.05), maintaining exploration for a longer period. This allows the agent to continue sampling diverse actions and experience a wider range of states.

As a result, DQN V2 solves the environment significantly faster, at episode 629. This represents an improvement of 318 episodes compared to DQN V1.

However, this increased exploration also introduces instability in later stages of training. Because the agent continues to take random actions even after learning a good policy, the final performance decreases, as reflected in the lower final score (182.07 compared to 217.39 in DQN V1).

This demonstrates an important limitation of epsilon-greedy exploration: while it improves learning speed, it can negatively affect final policy stability if not properly controlled.

Exploration in Policy-Based Methods (PPO)

In policy-based methods such as Proximal Policy Optimisation (PPO), exploration is handled differently. Instead of explicitly introducing random actions through an epsilon parameter, PPO encourages exploration using an entropy bonus.

Entropy measures the randomness of the action distribution produced by the policy. A higher entropy value indicates that the agent is selecting actions more uniformly, while lower entropy indicates more deterministic behaviour.

By adding an entropy term to the objective function, PPO encourages the policy to remain stochastic during training. This prevents the agent from prematurely converging to a deterministic policy and helps maintain exploration throughout the learning process.

Unlike epsilon-greedy strategies, entropy-based exploration does not require a manually defined decay schedule. Instead, exploration is automatically adjusted as part of the optimisation process.

Effect of Exploration in PPO V3

The results obtained in Problem 1 show that PPO V3 achieves both fast convergence and high stability.

PPO V3 solves the environment at episode 627, which is comparable to DQN V2. However, unlike DQN V2, PPO maintains stable performance throughout training and achieves a significantly higher final score of 252.35.

Furthermore, during evaluation, PPO V3 demonstrates superior consistency:

- 75% perfect landings
- 0 crashes across 20 episodes
- average score of 265.45

These results indicate that entropy-based exploration allows PPO to maintain an effective balance between exploration and exploitation. The agent continues to explore sufficiently during training while still converging to a stable and high-performing policy.

Comparison of Exploration Strategies

The comparison between DQN and PPO highlights key differences in how exploration is managed:

- DQN relies on externally controlled exploration through epsilon decay
- PPO integrates exploration directly into the learning objective via entropy

In DQN, exploration decreases over time regardless of the agent's performance. This can lead to premature exploitation or excessive randomness, depending on the chosen parameters.

In PPO, exploration is adaptive and naturally decreases as the policy becomes more confident. This results in smoother learning and improved stability.

Overall Discussion

The experiments conducted in Problem 1 clearly show that exploration strategy is a critical factor in reinforcement learning performance.

DQN V2 demonstrates that increasing exploration can significantly accelerate learning. However, it also shows that maintaining too much randomness can harm final performance.

PPO V3, on the other hand, achieves both fast convergence and high stability due to its entropy-based exploration mechanism. This suggests that policy-based methods are better suited for maintaining a consistent balance between exploration and exploitation.

Conclusion

Exploration and exploitation are fundamental components of reinforcement learning, and their balance directly affects both convergence speed and final performance.

The results of this project demonstrate that:

- insufficient exploration leads to slow learning (DQN V1)
- increased exploration improves convergence (DQN V2)
- adaptive exploration leads to superior performance and stability (PPO V3)

Overall, entropy-based exploration in PPO provides a more robust and effective approach compared to epsilon-greedy strategies in DQN.

Optional Extension: LLM-Based Chess Agent

Method

A chess-playing agent was implemented using a large language model accessed locally through Ollama. The model used was Qwen 2.5 7B, a general-purpose instruction-following model selected for its balance between reasoning capability and local execution speed.

The agent used the `python-chess` library to manage all game logic, including board representation, legal move generation, move validation, and game outcome detection. This ensured that the agent always operated within the rules of chess regardless of the model's output.

Prompt Design

At each turn, the agent constructed a prompt containing the following information:

- The current board state in FEN notation
- An ASCII board visualisation
- The complete list of legal moves in UCI format
- Strategic hints identifying available checks and captures, encouraging the model to prioritise aggressive and tactically relevant moves

The model was instructed to return a single move from the legal moves list with no additional explanation. Including the legal moves list directly in the prompt was the primary mechanism for ensuring move legality, as the model could only select from explicitly provided options.

Handling Invalid Moves

Despite being provided with the legal moves list, the model occasionally returned malformed or illegal responses. To handle this robustly, the following procedure was implemented:

- The model's response was parsed and cleaned (lowercasing, stripping whitespace, extracting the first token)
- The move was validated against the legal moves generated by `python-chess`
- If the move was invalid, the system retried up to three times
- If all three attempts failed, a random legal move was selected as a fallback

This approach ensured the game always continued without interruption while tracking the number of fallbacks as a diagnostic metric.

Evaluation

The LLM agent was evaluated against a random baseline agent over 20 games. The colour assignment was alternated so that the LLM played as White in 10 games and as Black in the remaining 10, ensuring a fair evaluation.

Results are presented in Figure 4.

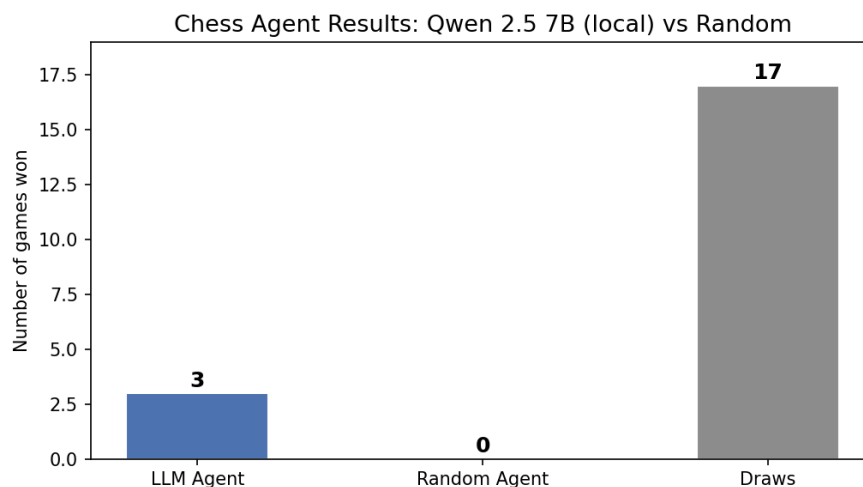


Figure 4: Evaluation results of the LLM-based chess agent against a random baseline over 20 games. The LLM agent achieved 3 wins (15%), the random agent achieved 0 wins, and 17 games (85%) resulted in draws.

Discussion of Results

The LLM agent achieved a win rate of 15% and recorded zero losses across all 20 games. The high draw rate of 85% is primarily explained by the `max_moves=250` limit, which caused many games to terminate due to move limits rather than checkmate. This is a known limitation when evaluating chess agents without a termination condition based on position repetition.

Despite the high draw rate, the results demonstrate that the LLM agent outperforms the random baseline. The agent won 3 games while the random agent won none, indicating that the model possesses a basic understanding of chess. The strategic hints included in the prompt, prioritising checks and captures, contributed to this advantage by guiding the model towards tactically relevant moves.

A GIF demonstrating a complete game where the LLM plays as White is included as a separate file (`chess_game_llm_white`) in the submission.

Limitations and Reproducibility

Several limitations affect the reproducibility of these results:

- **Non-determinism:** The Qwen 2.5 7B model does not produce deterministic outputs. Running the same evaluation may yield different results.
- **Fallback moves:** When the model failed to produce a valid move after three attempts, a random move was selected. This means a portion of the agent’s moves were effectively random, which may have contributed to the high draw rate.
- **Model size:** Qwen 2.5 7B is a relatively small model. Larger models with stronger reasoning capabilities would likely produce more decisive results and fewer fallbacks.
- **No search:** The agent does not use tree search or lookahead. All decisions are based solely on the current board state provided in the prompt.

A Benchmark Study: SOTA Deep Reinforcement Learning on Humanoid-v4

Introduction

The MuJoCo Humanoid-v4 environment [1, 2] is a high-dimensional continuous-control benchmark with a 376-dimensional proprioceptive observation space and 17 actuated degrees of freedom ($\mathcal{A} \subset [-0.4, 0.4]^{17}$). Episode reward is composed of a forward-velocity term, a per-step alive bonus of 5, and quadratic penalties on control effort and contact forces. The alive bonus creates a *reward-hacking attractor*: an agent can accumulate positive return simply by standing still, which makes the task a sensitive diagnostic for comparing algorithmic inductive biases.

Five structurally distinct algorithms are benchmarked, each trained for 1,000 episodes across five independent seeds: PPO [3], SAC [4], TD3 [5], TQC [6], and CrossQ [7]. A side-by-side qualitative comparison of all trained agents is available at <https://youtu.be/b4NnkzUpHzI>.

Algorithms

PPO [3] is an on-policy trust-region method that maximises a clipped surrogate objective:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon) \hat{A}_t \right) \right], \quad (1)$$

where $r_t(\theta)$ is the probability ratio and $\hat{A}_t$ is the Generalised Advantage Estimate ($\lambda=0.95$) [8]. Observations and rewards are normalised via `VecNormalize` [9].

SAC [4] is an off-policy max-entropy actor-critic that augments reward with a policy-entropy bonus: $r_t^{\text{aug}} = r_t + \alpha \mathcal{H}(\pi(\cdot|s_t))$. The temperature α is auto-tuned to maintain target entropy $\bar{\mathcal{H}} = -\dim(\mathcal{A}) = -17$ nats [10].

TD3 [5] extends DDPG with twin critics (minimum target), delayed policy updates ($d=2$), and target policy smoothing. Exploration uses fixed Gaussian action noise $\mathcal{N}(0, 0.1\mathbf{I})$, which is non-adaptive.

TQC [6] models the Q-value *distribution* with $N=5$ critics and $M=25$ quantiles each. Dropping the top $k=2$ atoms per critic before computing Bellman targets provides a principled pessimistic bias that attenuates bootstrap overestimation:

$$y = r + \gamma \frac{1}{N(M-k)} \sum_{n=1}^N \sum_{m=1}^{M-k} \hat{\theta}_{nm}(s', a'^*). \quad (2)$$

CrossQ [7] removes target networks entirely by incorporating BatchRenorm [11] into the critic. This shortens the TD information propagation path and yields superior sample efficiency. The critic uses a wider architecture ([2048, 2048]) versus [256, 256] for all other methods.

Experimental Setup

All experiments use Gymnasium 0.29.1 and Stable-Baselines3 ≥ 2.3 [9] on a single NVIDIA T4 GPU. Each (algorithm, seed) pair is trained for exactly 1,000 episodes via a stopping callback—fairer than a fixed timestep budget because episode length ranges from ≈ 26 steps (random policy) to $\approx 1,000$ steps (mature policy). Post-training evaluation uses 10 deterministic episodes per model; a 50-episode random-policy baseline anchors the outcome thresholds. Shared hyperparameters: $\gamma=0.99$; learning rate 3×10^{-4} (CrossQ: 10^{-3}); replay buffer 300,000 (off-policy); $\tau=0.005$.

Exploration and Exploitation

Classical ϵ -greedy exploration [12] is inapplicable here: it presupposes a finite discrete action set, whereas uniform random sampling over $\mathcal{A} \subset \mathbb{R}^{17}$ produces incoherent torque sequences that terminate episodes in ≈ 26 steps (as confirmed by our random baseline). Each algorithm instead uses a continuous-action-appropriate mechanism:

- **PPO**: exploration is intrinsic to the stochastic Gaussian policy. The learned variance $\sigma_\theta(s)$ is broad early in training and narrows as the policy converges; the trust-region clip prevents premature collapse.
- **SAC, TQC, CrossQ**: max-entropy exploration trains the policy to be appropriately stochastic at each state. The auto-tuned α is large early in training (encouraging broad exploration) and decays naturally—no manual schedule required.
- **TD3**: exploration relies entirely on fixed Gaussian noise. This non-adaptive mechanism cannot respond to the reward landscape and is the primary driver of TD3’s poor performance on this task.

On `Humanoid-v4` the alive bonus creates a strong local optimum (standing still). Adaptive high-entropy exploration helps SAC, TQC, and CrossQ escape it; TD3’s fixed noise and PPO’s trust-region do not provide sufficient diversity to do so consistently, as confirmed by the results below.

Results

Learning Progress

Figure 5 shows episode return versus episode index (0–1000, 5 seeds, mean $\pm 1\sigma$). All off-policy methods exceed the random baseline (mean 130.1) within the first 100 episodes. CrossQ demonstrates the steepest learning trajectory; PPO converges to a low plateau (≈ 109 mean return) characteristic of the alive-bonus attractor. Figure 6 shows the same data against cumulative timesteps.

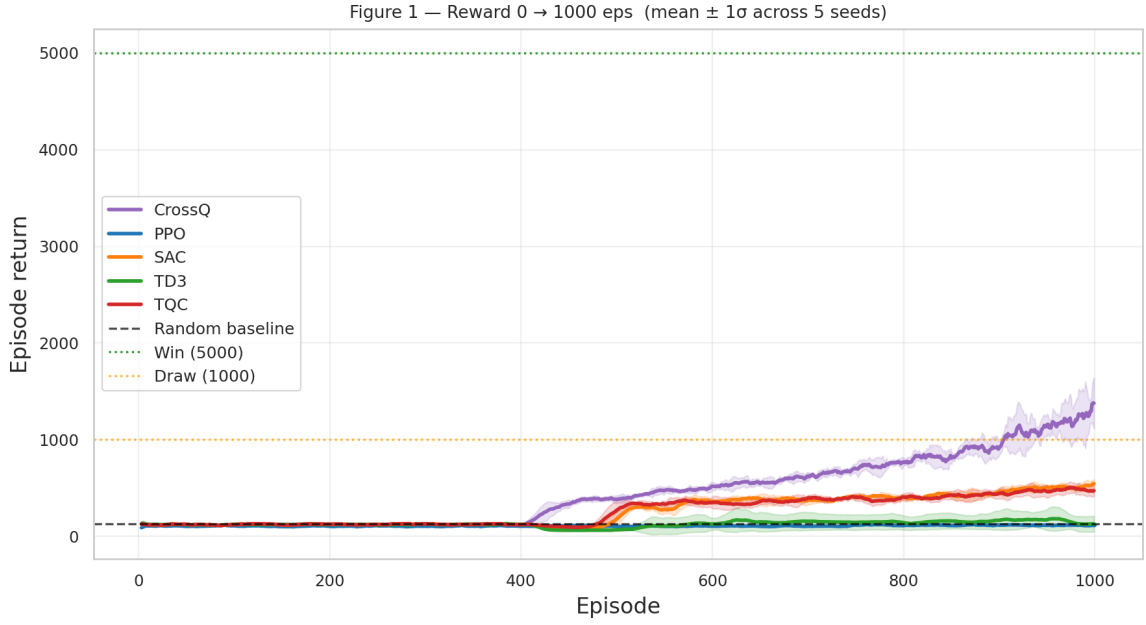


Figure 5: Episode return vs. episode index (mean $\pm 1\sigma$ across 5 seeds). Dashed line: random baseline (130.1). Dotted lines: Win (5000) and Draw (1000) thresholds.

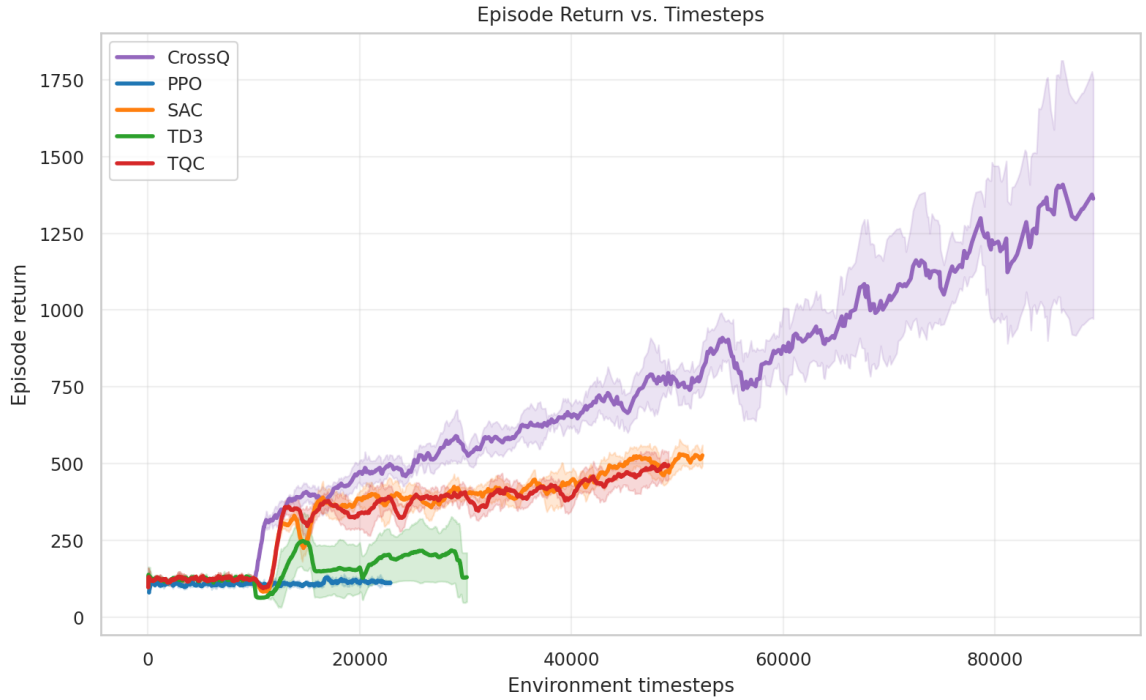


Figure 6: Episode return vs. cumulative environment timesteps (mean $\pm 1\sigma$).

Training Loss

Figure 7 shows actor and critic loss trajectories aligned to episode index. TQC’s truncated targets yield a lower stable critic loss than SAC at equal network width, consistent with reduced bootstrap error amplification. CrossQ exhibits higher initial variance (no target-network stabilisation) that resolves as BatchRenorm statistics converge.

Figure 2 — Training loss 0 → 1000 eps (mean $\pm 1\sigma$ across 5 seeds)

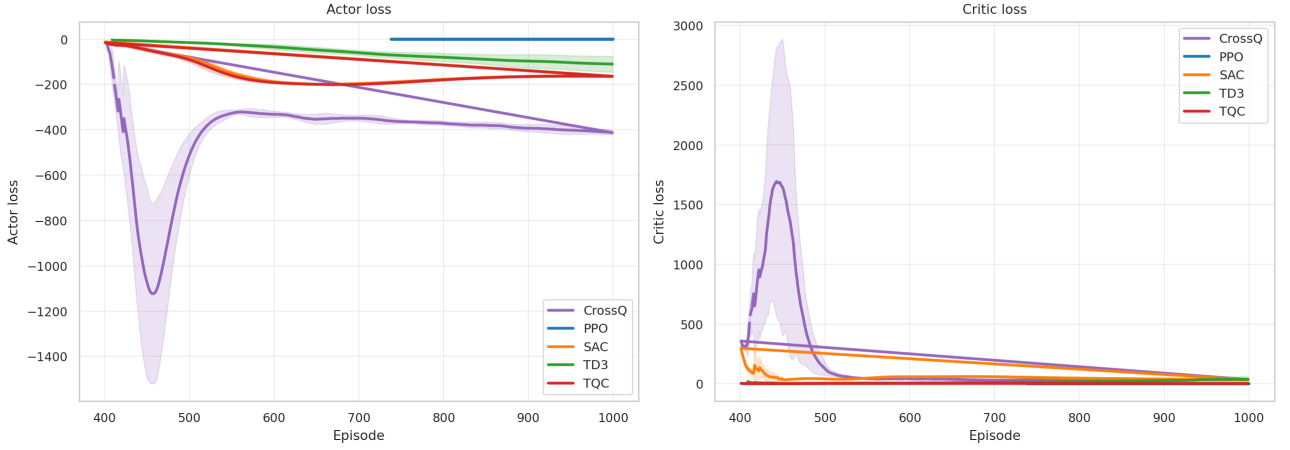


Figure 7: Training loss dynamics (mean $\pm 1\sigma$, aggregated to mean-per-episode and smoothed). Left: actor loss; right: critic loss.

Summary Statistics

Table 7 summarises the main results. CrossQ leads on all return and survival metrics. PPO’s large mean-to-max gap (109 vs. 311) reflects high inter-episode variance from the standing-still attractor. TD3’s poor sample efficiency despite the fewest timesteps per episode reflects the inadequacy of fixed Gaussian exploration.

Table 7: Main results: 1,000 episodes $\times$ 5 seeds on Humanoid-v4. Eval: mean deterministic return (10 episodes). Survival: mean episode length (steps).

Algorithm	Eval	Mean	Max	Survival	Wall (s)
CrossQ	1209.8	463.0	6036.6	87.5	1888
TQC	567.1	262.4	1065.5	52.1	785
SAC	498.5	265.0	1224.2	52.9	544
PPO	208.6	109.4	310.8	22.5	28
TD3	125.7	128.8	659.4	26.1	131

Sample Efficiency

Figure 8 plots return on a log-timestep axis. CrossQ achieves the highest return at every timestep budget, validating that removing target-network lag accelerates Q-value propagation.

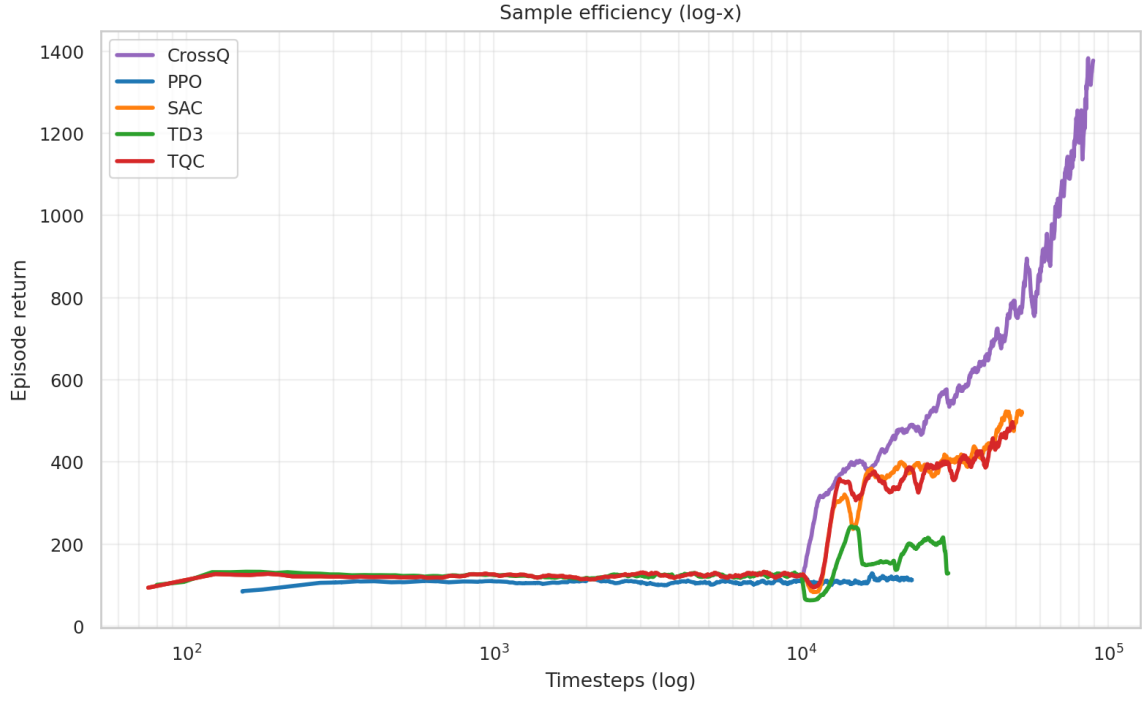


Figure 8: Sample efficiency: return vs. timesteps (log- x). Steeper curves indicate higher sample efficiency.

Locomotion Quality

Figures 9 and 10 show forward velocity, action smoothness ($\|\Delta a\|_2$; lower is smoother), torso height, and survival steps. Table 8 reports physics-state locomotion metrics from post-hoc evaluation.

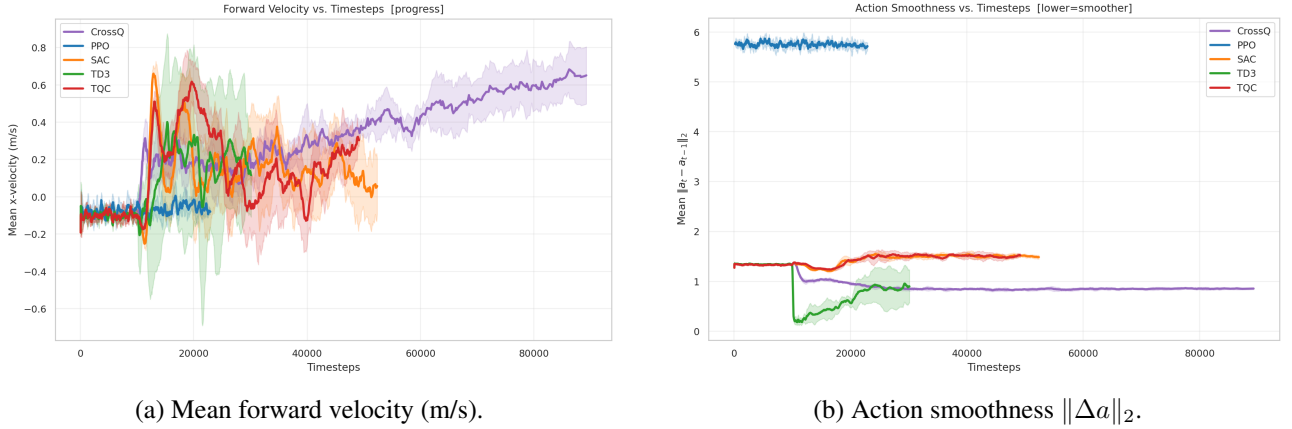


Figure 9: Locomotion quality metrics vs. timesteps (mean $\pm 1\sigma$).

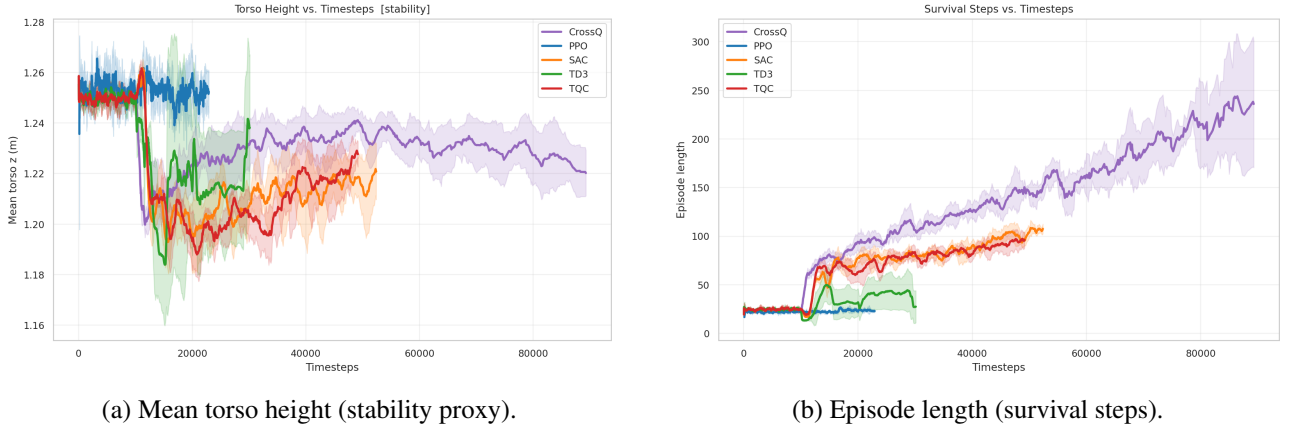


Figure 10: Additional locomotion metrics vs. timesteps.

PPO’s mean forward displacement is ≈ 0 m, confirming reward hacking: the policy maximises episode length by standing still. CrossQ achieves a mean $\Delta x = 3.28$ m and forward velocity 0.53 m/s across seeds (best seed: 6.31 m). PPO’s control cost ($\|a\|^2 = 17.01$) is an order of magnitude larger than all off-policy methods, reflecting uncoordinated torque patterns.

Table 8: Locomotion quality: post-hoc 5-episode evaluation (mean across 5 seeds). Δx : distance travelled; $\bar{v}$: mean forward velocity; Stand: fraction of steps with torso $z \in [1, 2]$ m.

Algorithm	Δx (m)	$\bar{v}$ (m/s)	Stand	$\ a\ ^2$
CrossQ	3.28	0.53	1.00	1.04
SAC	0.36	0.27	0.99	1.21
TQC	0.22	0.16	0.99	1.22
TD3	0.28	0.32	0.97	1.85
PPO	0.00	0.20	1.00	17.01

Win / Draw / Loss Outcome Rates

Outcome categories are defined by per-episode return: **Win** ($\geq 5,000$) denotes stable locomotion; **Draw** ($[1,000, 5,000)$) denotes partial locomotion without early termination; **Loss** ($< 1,000$) denotes falling or stagnation. The 1,000 boundary represents ≈ 200 steps at alive-bonus rate, a meaningful departure from the random-policy regime (mean 130.1, length 26 steps).

Figure 11 and Table 9 report outcome distributions across all 5,000 training episodes per algorithm. No algorithm achieves a Win within this 1,000-episode budget, consistent with the known difficulty of Humanoid-v4 at modest sample budgets [6, 10]. CrossQ is the only algorithm with a meaningful Draw rate (9.4%), confirming that its sample-efficiency advantage translates into qualitatively superior behaviour: sustained locomotion for 200+ steps in roughly 1 in 10 episodes, achieved from only $\approx 87,500$ total environment interactions.

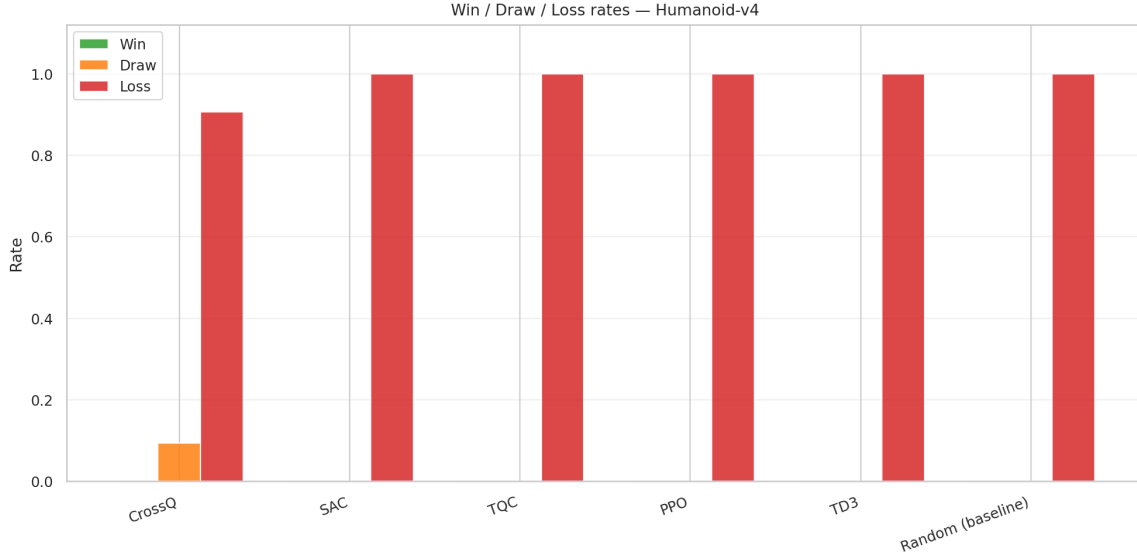


Figure 11: Win (≥ 5000), Draw ($[1000, 5000)$), and Loss (< 1000) outcome rates over 5,000 training episodes per algorithm. CrossQ is the only algorithm with a meaningful Draw rate (9.4%).

Table 9: Outcome rates (fraction of all training episodes) and mean return relative to the random-policy baseline.

Algorithm	Win	Draw	Loss	vs. Random (%)
CrossQ	0.000	0.094	0.906	+255.9
SAC	0.000	0.001	0.999	+103.7
TQC	0.000	0.000	1.000	+101.7
TD3	0.000	0.000	1.000	−1.0
PPO	0.000	0.000	1.000	−15.9
Random (baseline)	0.000	0.000	1.000	0.0

PPO and TD3 score below the random baseline (−15.9% and −1.0%). This is not paradoxical: both have learned to extend episodes by standing still, accumulating alive-bonus returns comparable to—or below—the random policy, while exhibiting qualitatively non-locomoting behaviour.

The side-by-side rollout video at <https://youtu.be/b4NnkzUpHzI> confirms these findings visually: CrossQ exhibits coherent forward gait; SAC and TQC show stepping behaviour that frequently collapses; TD3 produces high-frequency joint oscillations; PPO stands nearly stationary.

Ablation Studies

Three targeted ablations isolate the effect of individual components, each run over 500 episodes.

Ablation A — PPO $\pm$ VecNormalize (Figure 12). Without reward and observation normalisation, the alive bonus accumulates to $\sim 5,000$ in successful episodes, causing value-function loss to dominate gradient flow and destabilise training. Normalisation is therefore a prerequisite for stable PPO on Humanoid-v4, not merely a convenience.

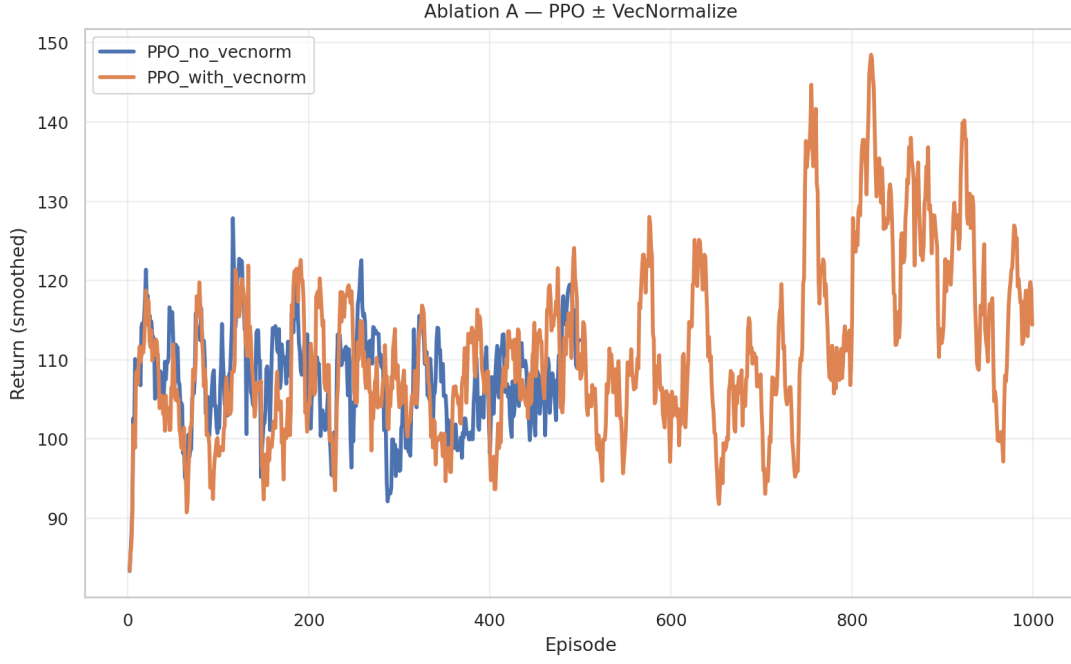


Figure 12: Ablation A: PPO with vs. without VecNormalize.

Ablation B — TQC quantile truncation $k \in \{0, 2, 5\}$ (Figure 13). Deterministic evaluation returns are 293.2, 300.0, and 301.8 for $k=0, 2, 5$ respectively. Differences are small at 500 episodes because overestimation bias has not yet compounded enough for a large separation; at scale the gap is expected to widen [6].

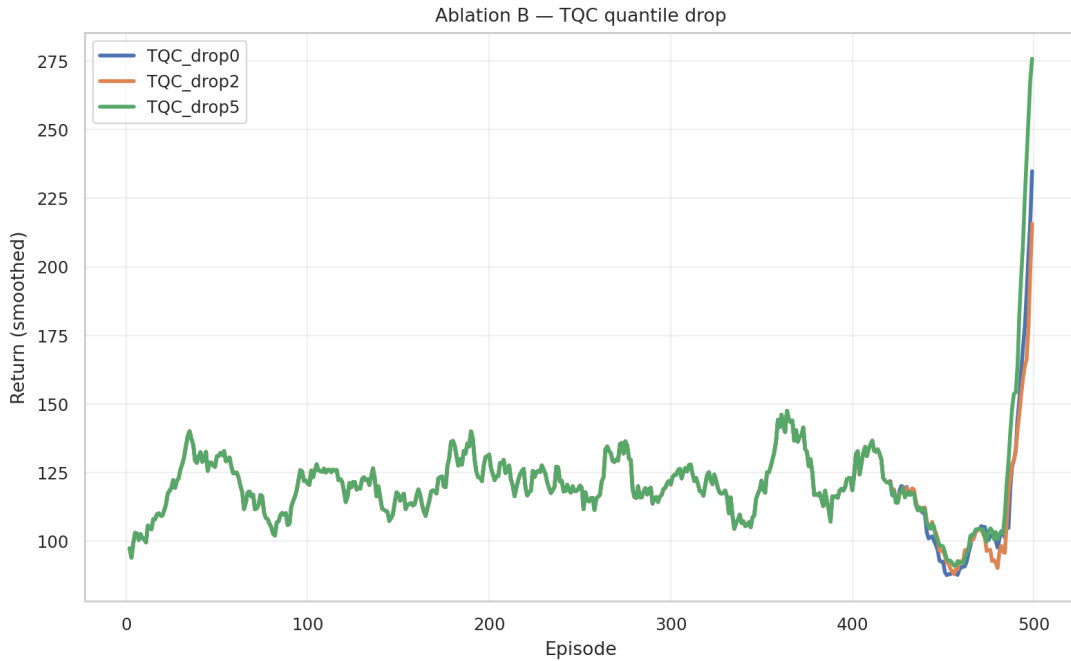


Figure 13: Ablation B: TQC with $k \in \{0, 2, 5\}$ dropped quantile atoms.

Ablation C — SAC entropy coefficient (Figure 14). At fixed $\alpha=0.005$, entropy regularisation is too weak to prevent premature exploitation, leading to entropy collapse. Auto-tuning is superior in all conditions, validating [10].

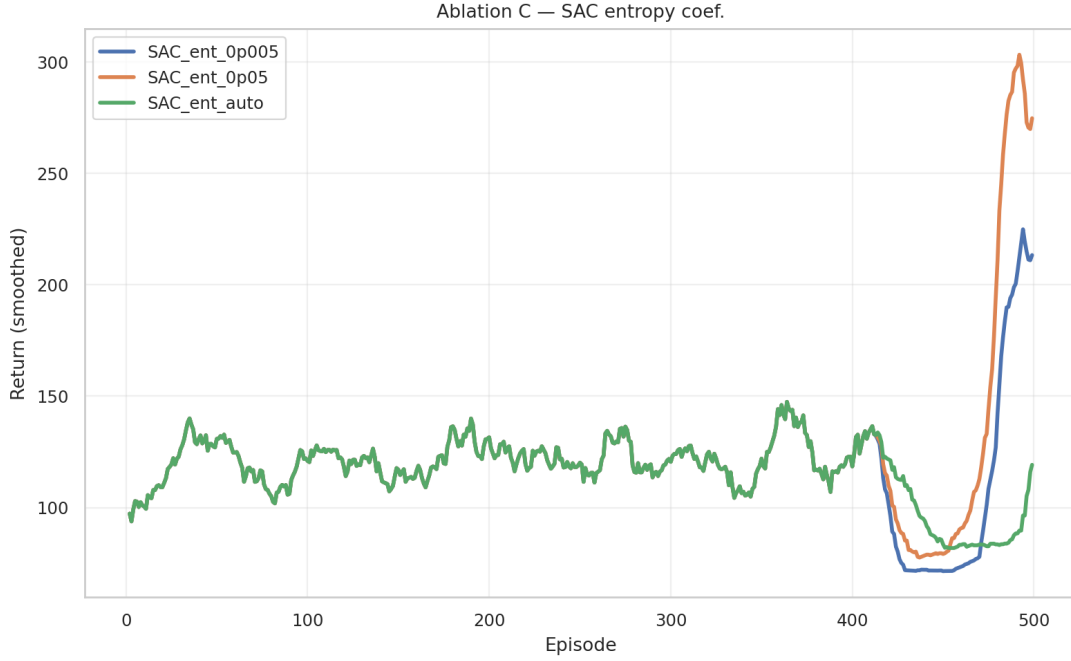


Figure 14: Ablation C: SAC with $\alpha \in \{\text{auto}, 0.05, 0.005\}$.

Discussion and Conclusion

CrossQ outperforms all methods across every metric at the 1,000-episode budget. Removing target networks shortens the bootstrap dependency chain: CrossQ’s critics receive immediate feedback rather than lagged updates at rate τ , accelerating value propagation. BatchRenorm [11] stabilises training without target averaging. The wider critic additionally provides representational capacity for the complex Humanoid Q-function.

TQC wins on deterministic evaluation (567.1) versus SAC (498.5), despite a slightly lower mean training return (262.4 vs. 265.0). This inversion is expected: pessimistic targets produce a more conservative, better-calibrated policy whose greedy mode outperforms at evaluation. At larger timestep budgets the TQC advantage is expected to grow [6].

PPO’s on-policy constraint limits it to $\approx 23,000$ total timesteps at 1,000 episodes — insufficient to escape the alive-bonus attractor. TD3’s fixed exploration noise cannot discover coherent locomotion strategies in the 17-dimensional joint space. The locomotion re-evaluation (Table 8) confirms that scalar reward alone is an insufficient performance metric for locomotion tasks: distance travelled and forward velocity, which cannot be reward-hacked, correctly diagnose the standing-still policy of PPO.

Future work should evaluate all methods at $\geq 10^6$ timesteps, where SAC, TQC, and CrossQ achieve robust walking behaviours [6, 10], and incorporate domain randomisation [13] to assess robustness relevant to sim-to-real transfer [14].

References

- [1] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012.
- [2] Mark Towers, Jordan K. Terry, Ariel Kwiatkowski, John U. Balis, Gianluca de Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Arjun KG, Markus Krimmel, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Andrew Tan Jin Shen, and Omar G. Younis. Gymnasium. 2023.
- [3] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. 2017.

- [4] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2018.
- [5] Scott Fujimoto, Herke van Hoof, and David Meger. Addressing function approximation error in actor-critic methods. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2018.
- [6] Arsenii Kuznetsov, Pavel Shvechikov, Alexander Grishin, and Dmitry Vetrov. Controlling overestimation bias with truncated mixture of continuous distributional quantile critics. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2020.
- [7] Aditya Bhatt, Daniel Palenicek, Boris Belousov, Max Argus, Artemij Amiranashvili, Thomas Brox, and Jan Peters. CrossQ: Batch normalization in deep reinforcement learning for greater sample efficiency and simplicity. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.
- [8] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. 2015.
- [9] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.
- [10] Tuomas Haarnoja, Aurick Zhou, Kristian Hartikainen, George Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, Henry Zhu, Abhishek Gupta, Pieter Abbeel, and Sergey Levine. Soft actor-critic algorithms and applications. 2018.
- [11] Sergey Ioffe. Batch renormalization: Towards reducing minibatch dependence in batch-normalized models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [12] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.
- [13] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017.
- [14] Jemin Hwangbo, Joonho Lee, Alexey Dosovitskiy, Dario Bellicoso, Vassilios Tsounis, Vladlen Koltun, and Marco Hutter. Learning agile and dynamic motor skills for legged robots. *Science Robotics*, 4(26), 2019.